

Pong (proper).

Paddles, ball deflection, AI, scenes.

Wire everything from Week 2 into one playable game.

RECAP

Where you left off.

Days 8–12 built every piece Pong needs. Today is the assembly step — no new concepts, just wiring.

Day 8–9

Loop + Sprites

60 FPS loop, event queue, Sprite/Group pattern. Ball already moves and bounces.

Day 10

Collisions

Rect.colliderect, group-vs-group. Brick-breaker proved paddle-ball collision works.

Day 11

Sound + animation

pygame.mixer for SFX. Today: hit sound on paddle/wall, score sound on point.

Day 12

Scene state machine

Scene base class with `handle_event` / `update` / `draw`. Menu → Play → GameOver.

What Pong is made of.

Before any code: list the moving parts. Pong has four — two paddles, one ball, a scoreboard. Each gets its own Sprite class. The scene from Day 12 owns them and runs the loop.

```
# pong.py — overall shape
import pygame, random
from scene import Scene # from Day 12

WIDTH, HEIGHT = 800, 600
WIN_SCORE = 11

class Paddle(pygame.sprite.Sprite): ...
class Ball(pygame.sprite.Sprite): ...
class PlayScene(Scene): ...
class MenuScene(Scene): ...

if __name__ == "__main__":
    run(MenuScene()) # scene runner from Day 12
```

Why four classes

Each thing on screen that moves or holds state = one Sprite subclass. The Scene owns the Groups and the score. The runner (Day 12) just calls `handle_event` / `update` / `draw` — it doesn't know it's running Pong.

MenuScene returns a PlayScene when SPACE is pressed. PlayScene returns a GameOverScene when a side hits WIN_SCORE.

The Paddle sprite.

A paddle is a tall rectangle with a y position, a speed, and a clamp so it doesn't leave the screen. Same Sprite pattern as Day 9 — `image + rect + update()`.

```
class Paddle(pygame.sprite.Sprite):
    def __init__(self, x, color=(255,255,255)):
        super().__init__()
        self.image = pygame.Surface((12, 90))
        self.image.fill(color)
        self.rect = self.image.get_rect(center=(x, HEIGHT//2))
        self.vy = 0
        self.speed = 420 # px / sec

    def update(self, dt):
        self.rect.y += int(self.vy * dt)
        self.rect.clamp_ip(SCREEN_RECT) # stay on screen
```

Anatomy

- `image` — a 12×90 white surface (just a filled rect)
- `rect` — the position. Pygame collisions all run off this.
- `vy` — vertical velocity in px/sec. Multiply by `dt` in `update` so speed is frame-rate independent.
- `clamp_ip` — pygame's built-in 'snap this rect inside another'. Cheapest way to keep paddles on screen.

Paddle-angle deflection.

This is the only real 'physics' in Pong. A pure x-flip on hit is boring — the angle should depend on WHERE on the paddle the ball lands. Hit the top of the paddle, ball flies up. Hit the middle, ball goes mostly horizontal.

```
def bounce_off_paddle(ball, paddle):  
    # offset: -1 (top of paddle) ... +1 (bottom)  
    offset = (ball.rect.centery - paddle.rect.centery) / (paddle.rect.height / 2)  
    offset = max(-1.0, min(1.0, offset))  
  
    # bounce angle: ±60° from horizontal  
    angle = offset * math.radians(60)  
    speed = min(ball.speed * 1.05, MAX_SPEED) # speed up a bit  
  
    direction = 1 if ball.vx < 0 else -1 # reflect  
    ball.vx = direction * speed * math.cos(angle)  
    ball.vy = speed * math.sin(angle)  
    ball.speed = speed
```

What's happening

offset maps the hit position to [-1, +1]. Scale that by 60° → bounce angle.

vx flips sign (ball reverses horizontal direction). vy comes from sin(angle), so a center hit (offset≈0) gives vy≈0.

Speed cap stops it going faster than the paddles can react. The 1.05 multiplier rewards long rallies — escalating pace.

Score and reset the ball.

When the ball leaves the screen on the left or right, the *OTHER* side scores. Reset to center with a random direction. First to *WIN_SCORE* wins.

```
class PlayScene(Scene):
    def update(self, dt):
        self.paddles.update(dt)
        self.ball.update(dt)

        # ball off left side → right player scores
        if self.ball.rect.right < 0:
            self.score_r += 1
            self._reset_ball(direction=-1)
        elif self.ball.rect.left > WIDTH:
            self.score_l += 1
            self._reset_ball(direction=1)

        if max(self.score_l, self.score_r) >= WIN_SCORE:
            self.next_scene = GameOverScene(self.score_l, self.score_r)
```

Reset details

`_reset_ball` puts the ball back at center, picks a random `vy` in a small range, and serves toward the player who just got scored on (direction argument).

`next_scene` is the Day 12 hook — Scene base class checks it after every frame and swaps when it's not `None`.

A right paddle that's beatable.

Perfect AI is boring — it never misses. The trick is a deadzone (don't move for tiny errors) plus a slower max speed than the player. Optionally, only track when the ball is moving toward the AI.

```
class AIPaddle(Paddle):
    def __init__(self, x, ball):
        super().__init__(x)
        self.ball = ball
        self.speed = 340 # slower than player (420)
        self.deadzone = 18 # px tolerance

    def update(self, dt):
        # only chase when ball moves toward us
        if self.ball.vx > 0:
            diff = self.ball.rect.centery - self.rect.centery
            if abs(diff) > self.deadzone:
                self.vy = self.speed if diff > 0 else -self.speed
            else: self.vy = 0
        else: self.vy = 0
        super().update(dt)
```

Difficulty knobs

- speed — lower = easier. Try 280 for casual, 380 for hard.
- deadzone — larger = jitterier, more beatable. 18 px is a good start.
- 'only chase when ball.vx > 0' — AI rests on its own serves. Without this, it tracks even when the ball is going away, which looks robotic.

Continuous vs event input.

Two ways to read the keyboard. *event queue (KEYDOWN / KEYUP) is for discrete actions — pause, fire, menu select.*
pygame.key.get_pressed() is for held keys — paddle movement.

```
# in PlayScene.handle_event — discrete (one-shot)
def handle_event(self, event):
    if event.type == pygame.KEYDOWN and event.key == pygame.K_p:
        self.paused = not self.paused

# in PlayScene.update — continuous (held key)
def update(self, dt):
    keys = pygame.key.get_pressed()
    self.player.vy = 0
    if keys[pygame.K_w]: self.player.vy = -self.player.speed
    if keys[pygame.K_s]: self.player.vy = self.player.speed
```

Rule of thumb

If you want the action to repeat while the key is held
→ `get_pressed()` in `update()`.

If you want it to fire exactly once per press →
`KEYDOWN` in `handle_event()`.

Setting `vy = 0` at the top of `update` means letting go
of a key naturally stops the paddle. No need to handle
`KEYUP` separately.

Pong tuning knobs.

Reasonable starting values for a 800×600 game. Tweak after first playthrough.

Knob	Value	Why	When to change
WIN_SCORE	11	Long enough to feel like a match, short enough to finish	Down to 5 for quick demos
Player speed	420 px/s	Just fast enough to react to top-bin balls	Up if paddle feels sluggish
AI speed	340 px/s	Slower than player → AI loses to corners	Up = harder, down = easier
Max bounce	60°	Steep enough to set up angles, not so steep ball goes vertical	Lower for floatier feel



TAKEAWAY

Hard-code these as constants at the top of pong.py. The first run will feel off — tweak two or three values, not all of them at once.

Build pong.py end-to-end.

MenuScene → PlayScene → GameOverScene. W/S for left paddle. AI on the right. First to 11.

STRUCTURE

- pong.py imports scene runner from Day 12
- Constants at top (WIDTH, HEIGHT, WIN_SCORE)
- Sprite Groups for paddles + ball
- Reuse Day 11 mixer for SFX

CLASSES

- Paddle (Sprite)
- AIPaddle (Paddle)
- Ball (Sprite)
- MenuScene / PlayScene / GameOverScene

BEHAVIOR

- Paddle-angle deflection ($\pm 60^\circ$ from offset)
- Speed-up multiplier capped at MAX_SPEED
- Reset serves toward last-scored-on side
- SPACE in menu → play. ENTER on game-over → menu.

Things that will trip you up.

Read these before you start coding. Two are gotchas (orange), two are advice (blue).

Ball clips into paddle on fast frames

If the ball is moving $>$ paddle width per frame, colliderect fires but the ball is already inside. After bounce, reposition ball.rect outside the paddle (ball.rect.right = paddle.rect.left) to avoid double-hits.

Diagonal speed is faster than horizontal

If you set $vx = \pm \text{speed}$ and add random vy on reset, total speed is $\sqrt{vx^2 + vy^2} > \text{speed}$. Normalize the velocity vector to a target speed instead.

Frame-rate independence with dt

Always multiply velocities by dt (seconds since last frame). $dt = \text{clock.tick}(60) / 1000$. Without this, the game plays differently on different machines.

Score font, not Surface.fill

Render the score with `pygame.font.Font.render()` once per score change, cache the surface. Re-rendering text every frame is fine for now but slow at scale.

End of Week 2.

After today: you've shipped a real game. Tomorrow is the stretch.

DAY 14

Sun May 24

Stretch pick — Asteroids-lite (vector physics, rotation, wraparound) OR brick-breaker polish with a JSON level loader. Decide tonight.

WEEK 3

May 25 - 31

Comprehensions, generators, decorators, lambda/map/filter. Then pyautogui + keyboard + subprocess. Project: CLI autoclicker with global hotkeys.

CRUX-LITE

End of June

Everything you've built feeds into the audio brain at the end of Phase 1. Pong proves you can wire a multi-class system end-to-end.

Ship pong.py today. Polish tomorrow. Phase 1 stays on track.